Buổi 7

SSO và chiến lược kiểm thử hệ thống

OIDC + PKCE · ID token và access token · logout · SPA hay BFF · tháp kiểm thử · chất lượng assertion

Mục tiêu buổi học

Sau buổi này, học viên có thể:

1. Giải thích luồng **Authorization Code + PKCE**: redirect, callback, đổi code lấy token, `state`, `nonce`, logout
2. Phân biệt **ID token** (cho app biết ai đăng nhập) và **access token** (cho API); biết vì sao API từ chối ID token
3. So sánh **SPA giữ token** với **BFF**; chọn theo rủi ro và hạ tầng
4. Xây **tháp kiểm thử**: unit test quy tắc, integration test HTTP + state, E2E hành trình nghiệp vụ
5. Đo **chất lượng assertion** bằng cách thử phá; viết **regression test** cho lỗi đã sửa

Sản phẩm đầu ra: checklist SSO · báo cáo kiểm thử · regression test

Lịch buổi học (4 giờ)

Thời gian	Hoạt động
20'	Review buổi 6 (ma trận bảo mật, vòng đời phiên), phân tích use case
40'	OIDC, PKCE, hai loại token, logout, SPA và BFF; tháp kiểm thử, assertion
30'	Live demo: đăng nhập Keycloak lỗi 401, máy dùng chung, thử phá test
10'	Nghỉ
120'	Lab 5 bài
20'	Review chéo, chốt checklist SSO và báo cáo kiểm thử

Review buổi 6

- Access token ngắn hạn + refresh token **rotation**: gửi lại refresh token đã dùng thì sao?
- Vì sao `share()` + `finalize` mà không `shareReplay(1)`?
- `startsWith(apiUrl)` gửi Bearer tới đâu? `apiPathOf` sửa thế nào?
- Ma trận bảo mật có dòng `agent1` gọi `/approve` → 403. **Còn API nào khác đổi trạng thái phiếu mà ma trận chưa thử?**

> Giữ câu hỏi cuối. Hôm nay sẽ có câu trả lời, và nó không dễ chịu.

Use case: VNPT muốn đăng nhập một lần (SSO)

Điểm bắt đầu `session07-start`: portal có chế độ `auth.mode = "oidc"`, thư viện `angular-auth-oidc-client`, Keycloak cục bộ (profile `sso`).

#	Triệu chứng	Đã đo
1	Đăng nhập Keycloak xong, **mọi API 401**	`The aud claim is not valid`; token gửi đi có `typ: ID`
2	Quầy dùng máy chung: đăng xuất, người sau bấm SSO **vào thẳng tài khoản người trước**	`GET /auth` → 302 ngay, không hiện form
3	**Agent đóng được phiếu**, bỏ qua Manager duyệt	`PATCH status=CLOSED` → **200** (lỗi có từ buổi 2)

Và: cố ý phá 4 chỗ trong luồng duyệt phiếu, bộ test "toàn xanh" **để lọt 2 chỗ**.

Phần 1

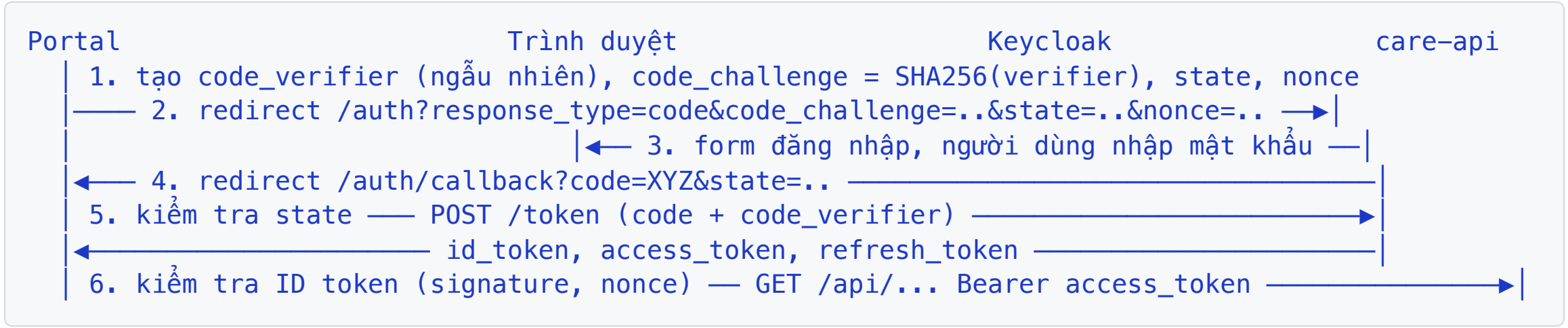
OpenID Connect cho SPA

Bốn vai trong OIDC

Vai	Care Portal	Giữ gì
Người dùng + trình duyệt	Nhân viên CSKH	Cookie phiên ở Keycloak
Client (Relying Party)	Portal Angular, client **public** `care-portal`	Token trong sessionStorage. **Không** có secret
Authorization Server (OpenID Provider)	Keycloak, realm `care`	Mật khẩu, phiên SSO, private key ký token
Resource Server	`care-api` (Spring)	Public key (JWKS), kiểm tra `iss`, `aud`, hạn

> App **không bao giờ** thấy mật khẩu SSO. Password grant (app tự gửi mật khẩu tới Keycloak) đã bị tắt: `Client not allowed for direct access grants`.

Authorization Code + PKCE



- `code` đi qua URL (lộ được: lịch sử, log proxy) nhưng **vô dụng nếu không có `code_verifier`**, thứ chỉ nằm trong tab
- Client public không giữ được secret → PKCE thay cho client secret

PKCE, state, nonce: mỗi thứ chặn một kiểu tấn công

Tham số	Chặn	Đã thử trên Keycloak
`code_challenge` + `code_verifier` (S256)	Ai đó lấy được `code` rồi tự đổi token	Thiếu: `Missing parameter: code_challenge_method`. `plain`: `not matching the configured one`
`state`	Login CSRF: kẻ gian gửi link callback chứa `code` của **họ**	Mở `/auth/callback?code=abc&state=gia`: thư viện không gọi `/token`, về `/login`
`nonce`	Dùng lại ID token cũ (replay)	Thư viện so `nonce` trong ID token với giá trị đã lưu
Redirect URI **chính xác**	Gửi `code` về trang của kẻ gian	`redirect_uri=http://evil.example/cb` → 400 "Tham số không hợp lệ"

> Wildcard `http://localhost:4200/*` trong redirect URI tiện lúc dev nhưng mở đường cho mọi trang cùng origin nhận `code`.

ID token hay access token?

Cùng một lần đăng nhập `agent1`, giải mã thật:

Claim	ID token	Access token
`typ`	`ID`	`Bearer`
`aud` (gửi cho ai)	`care-portal` (**app**)	`care-api` (**API**)
Có riêng	`nonce`, `at_hash`	`scope`, `realm_access`, `allowed-origins`
Dùng để	App biết **ai** đăng nhập: tên, vai trò để hiển thị	Gọi API. App coi như **hộp kín**

```
GET /api/subscribers  Authorization: Bearer <ID token>
401  WWW-Authenticate: Bearer error="invalid_token", error_description="... The aud claim is not valid"
```

Lỗi 1 nằm ở đâu

```
// oidc-auth.service.ts (điểm bắt đầu)
private apply(result: LoginResponse): void {
  ...
  this.token = result.idToken;                                  // token gửi cho API
  this.user.set(userFromClaims(decodeJwtPayload(this.token)));  // tên, vai trò
}
```

- Một biến cho hai việc: tiện, vì ID token có sẵn tên và vai trò
- Interceptor gặp 401 → refresh → **gửi lại cũng ID token** → lại 401. Người dùng thấy "Không tải được danh sách"
- Server làm đúng: kiểm `aud` là thứ chặn được lỗi này. API nào bỏ qua `aud` sẽ chấp nhận cả ID token

> Sửa: `this.token = result.accessToken`; tên, vai trò đọc từ `result.idToken`. Test: header `Authorization` phải là access token.

Một port, hai adapter (như `TicketPort` buổi 1)

```
// port: guard, interceptor, màn hình chỉ biết lớp này
export abstract class AuthService {
  abstract readonly mode: 'local' | 'oidc';
  abstract init(): Promise<void>;
  abstract accessToken(): string | null;
  abstract refresh(): Observable<string>;
  abstract loginWithSso(returnUrl: string): void;
  abstract logout(): void;
}

// composition root: chọn adapter theo config.json
export function provideAuth(config: AuthConfig = { mode: 'local' }) {
  return config.mode === 'oidc'
    ? [provideOidcClient({ ... }), { provide: AuthService, useExisting: OidcAuthService }]
    : [{ provide: AuthService, useExisting: LocalAuthService }];
}
```

Đổi cách đăng nhập = sửa `config.json`, không build lại. Không feature nào phải sửa.

Phía API: nhận token của hai nơi phát

```
// JwtConfig: chọn decoder theo claim "iss"; issuer lạ -> 401 ngay
managers.put("care-api", manager(localDecoder));                          // HS256, buổi 6
managers.put("http://localhost:8180/realms/care", manager(ssoDecoder));   // RS256, Keycloak
return new JwtIssuerAuthenticationManagerResolver(managers::get);

static JwtDecoder ssoDecoder(SsoProperties sso) {
  var decoder = NimbusJwtDecoder.withJwkSetUri(sso.jwkSetUri()).build();  // public key qua HTTP
  decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
      new JwtTimestampValidator(CLOCK_SKEW), new JwtIssuerValidator(sso.issuerUri()),
      new JwtClaimValidator<List<String>>("aud", aud -> aud != null && aud.contains("care-api"))));
```

- `iss` = URL trình duyệt thấy (`localhost:8180`); JWKS lấy qua mạng docker (`keycloak:8180`)
- Vai trò: mapper `roles` trong realm → `RolesClaimConverter` của buổi 6 dùng lại nguyên

Đăng xuất: xóa token trong tab chưa phải đăng xuất

Điểm bắt đầu: `logoffLocal()`

- Xóa token trong sessionStorage
- Cookie phiên ở `localhost:8180` **còn nguyên**
- Người sau bấm SSO → `/auth` trả **302** ngay → vào tài khoản cũ

RP-initiated logout

- `POST /revoke` refresh token, access token
- Chuyển tới `end_session_endpoint` với `id_token_hint`, `post_logout_redirect_uri`
- Keycloak xóa phiên → về `/login`

> Phiên SSO dài hơn phiên của app: đăng xuất ở app phải kết thúc phiên ở máy chủ định danh, nhất là máy dùng chung tại quầy giao dịch.

SPA giữ token hay BFF?

	SPA (Care Portal hiện tại)	BFF (Backend for Frontend)
Token ở đâu	sessionStorage của tab	Server BFF; trình duyệt chỉ có cookie `HttpOnly`
XSS đọc được token	**Có**	Không (nhưng XSS vẫn gọi API qua cookie)
CSRF	Không (Bearer không tự gửi)	**Phải chống**: `SameSite`, token CSRF
Hạ tầng	Không thêm	Thêm một server, session store, cấu hình proxy
Client	Public + PKCE	**Confidential** (có secret, ở server)

> Mạng nội bộ, phiên ngắn, CSP chặt: SPA + PKCE chấp nhận được. Ra Internet, dữ liệu khách hàng nhạy cảm: cân nhắc BFF. Ghi quyết định vào ADR.

Phần 2

Chiến lược kiểm thử hệ thống

Tháp kiểm thử của Care Portal (số thật)

Tầng	Công cụ	Số test	Thời gian	Kiểm cái gì
Unit: quy tắc	Vitest, hàm thuần	9 (`canApprove`)	< 1 giây	Bảng trạng thái × vai trò
Integration: HTTP + state	TestBed, `HttpTestingController`	9 OIDC + 3 duyệt phiếu	1–2 giây	Header, request, store, màn hình
Toàn bộ frontend	`nx run-many -t test`	143		
API trực tiếp	Playwright `request`	20	1,6 giây	401/403/409 và **dữ liệu không đổi**
E2E hành trình	Playwright + trình duyệt	1 duyệt phiếu, 4 SSO	2,4 và 6,7 giây	Đi đúng đường người dùng đi

> Quy tắc nghiệp vụ test ở tầng rẻ nhất có thể. E2E chỉ cho hành trình quan trọng: chậm hơn, cần backend, khó chỉ ra chỗ hỏng.

Unit test quy tắc: hàm thuần + bảng

```
// libs/portal/domain-tickets/src/lib/ticket.rules.ts (không phụ thuộc Angular)
export function canApprove(ticket: Pick<TicketSummary, 'status'>, roles: readonly string[]) {
  return roles.includes('SUPERVISOR') && ticket.status === 'RESOLVED';
}
```

```
it.each<[TicketStatus, string[], boolean]>([
  ['RESOLVED', ['AGENT', 'SUPERVISOR'], true],
  ['RESOLVED', ['AGENT'], false],
  ['RESOLVED', [], false],
  ['NEW', ['SUPERVISOR'], false],
  ['CLOSED', ['SUPERVISOR'], false],
  // ...
])('phieu %s, vai tro %j -> %s', (status, roles, expected) =>
  expect(canApprove({ status }, roles)).toBe(expected));
```

Trước đây quy tắc nằm trong template (`isManager() && t.status === 'RESOLVED'`): chỉ E2E mới thử được.

Integration test: route thật + store + adapter HTTP thật

```
TestBed.configureTestingModule({ providers: [
  provideRouter(appRoutes),                          // route của app, kể cả provideHttpTicketPort()
  provideHttpClient(), provideHttpClientTesting(), // chỉ thay backend
  { provide: AuthService, useValue: fakeSupervisor },
]});
const harness = await RouterTestingHarness.create('/tickets?status=RESOLVED');
http.expectOne((r) => r.url === `${API}/api/tickets`).flush(page([resolved]));
```

- Thật: route, guard, trang, `TicketListStore`, `HttpTicketAdapter`, xử lý lỗi 409
- Giả: server (`HttpTestingController`), người đăng nhập
- Đặt ở `apps/portal`: nơi nối mọi thứ (lib feature bị cấm import `@angular/common/http`)

Ba kiểu assertion "xanh mà vô dụng"

```
const req = http.expectOne((r) => r.url.endsWith('/approve'));
req.flush({ ...resolved, status: 'CLOSED' });
expect(req.request.method).toBe('POST');                  // 1. kiểm thứ hiển nhiên, bỏ qua body
```

```
expect(el.querySelector('nz-alert')).toBeTruthy();        // 2. "có alert", không biết alert nói gì
```

```
roles.set(['AGENT']);
const harness = await RouterTestingHarness.create('/tickets?status=RESOLVED');
expect(approveButtons(el)).toHaveLength(0);                // 3. danh sách chưa từng được trả về:
                                                           //    0 nút là đương nhiên
```

> Test không thể đỏ thì không bảo vệ được gì. Cách duy nhất để biết: **cố ý phá code** và xem test có đỏ không.

Thử phá (mutation bằng tay): kết quả thật

Phá	39 test `feature-tickets`	3 test app (yếu)	3 test app (viết lại)	9 test quy tắc
M1 gửi `version: 0`	2 đỏ	xanh	đỏ	xanh
M2 bỏ `store.reload()`	1 đỏ	xanh	đỏ	xanh
M3 409 → "lỗi chung"	**xanh**	xanh	đỏ	xanh
M4 mọi vai trò thấy nút Duyệt	**xanh**	xanh	đỏ	3 đỏ

Viết lại: kiểm URL và body `{ version: 3 }`, request tải lại đúng bộ lọc, chữ "Đã duyệt phiếu TK-000042", lý do 409 từ server, danh sách **đã hiện** trước khi đếm nút, `http.verify()`.

> Công cụ tự động (Stryker) làm việc này hàng loạt. Bằng tay 4 chỗ đã đủ để thấy lỗ hổng.

E2E hành trình: chọn đúng việc cho E2E

```
// 1. Agent tạo phiếu trên giao diện, lấy id/code từ response
// 2. Arrange qua API: PATCH status RESOLVED (giao diện chưa có nút này)
// 3. Manager duyệt trên giao diện, đúng dòng có code
// 4. Agent mở /tickets?status=CLOSED: dòng có code hiện "Đã đóng"
// 5. GET /api/tickets/{id}: status CLOSED, version + 2
```

- Hai `browser.newContext()`: hai người, hai phiên, cùng lúc
- Bước phụ đi qua API cho nhanh và ổn định; bước nghiệp vụ chính đi qua UI
- Kiểm cả **màn hình** lẫn **dữ liệu ở server**
- Bẫy thật: đọc `response.json()` sau khi trang đã chuyển → `No data found for resource`

Test quyền phải đi thẳng vào API

```
const res = await request.patch(`/api/tickets/${ticket.id}`, {
  headers: { Authorization: agentToken },
  data: { status: 'CLOSED', version: ticket.version },
});
expect(res.status()).toBe(403);       // Expected: 403  Received: 200
```

- Lỗi có từ buổi 2. Giao diện không có nút đóng phiếu → mọi test qua UI xanh suốt 5 buổi
- Ma trận buổi 6 chỉ thử `/approve`. Hỏi đúng câu: "**mọi** API đổi trạng thái phiếu là gì?"
- Sửa: `@PreAuthorize("#request.status() != T(...TicketStatus).CLOSED")`: đóng phiếu chỉ qua `/approve`

> Regression test ở hai tầng: `TicketApiTests.nobodyClosesTicketByPatch` (backend) và `role-denial.spec.ts` (API).

Regression test và báo cáo kiểm thử

Regression test tốt:

- **Đỏ trước** trên code lỗi (ghi lại thông báo đỏ), xanh sau khi sửa
- Kiểm hành vi người dùng thấy hoặc dữ liệu, không kiểm chi tiết cài đặt
- Đặt ở tầng **rẻ nhất** tái hiện được lỗi; tên test nói lỗi gì

Báo cáo kiểm thử (`docs/bao-cao-kiem-thu.md`):

Phạm vi rủi ro	Tháp: tầng, số test, thời gian	Lỗi tìm được + bằng chứng	Thử phá trước/sau	Rủi ro còn lại

Live demo (30 phút)

1. `docker compose --profile sso up -d`, đổi `config.json` sang OIDC, đăng nhập `agent1`: **401**. Giải mã hai token trong DevTools
2. Đăng xuất, bấm SSO: **vào lại không hỏi mật khẩu**. Xem cookie ở `localhost:8180`
3. `curl` thử redirect URI lạ, bỏ PKCE, password grant: Keycloak từ chối thế nào
4. `ticket-approval.spec.ts`: 3 test xanh. Phá M3 → vẫn xanh. Phá M4 → vẫn xanh
5. `curl -X PATCH ... '{"status":"CLOSED"}'` bằng token agent: **200**

Lab (120 phút)

Bài	Nội dung	Phút
1	Đăng nhập Keycloak, giải mã hai token; test đỏ "API nhận access token", sửa adapter	35
2	Đăng xuất máy dùng chung (RP-initiated logout) + `docs/checklist-sso.md`	20
3	`canApprove` + test bảng; viết lại integration test; thử phá M1–M4	30
4	Playwright: hành trình Agent → Manager → Agent; test API gọi thẳng → tìm lỗi PATCH	25
5	Sửa backend (một dòng), regression test, `docs/bao-cao-kiem-thu.md`	10

Chuẩn bị: `docker compose -f infra/docker-compose.yml --profile sso up -d --build` · `npx playwright install chromium` · portal ở cổng 4200

Chốt buổi

- PKCE thay client secret cho SPA; `state`, `nonce`, redirect URI chính xác mỗi thứ chặn một kiểu tấn công
- **ID token cho app, access token cho API**; API kiểm `aud` thì lỗi lộ ra ngay
- Đăng xuất SSO = kết thúc phiên ở máy chủ định danh, không chỉ xóa token
- Quy tắc ở unit, nối dây ở integration, hành trình ở E2E, quyền ở **API trực tiếp**
- Test chỉ đáng tin khi đã thấy nó **đỏ**: thử phá trước khi tin

Bài tập về nhà: `course/bai-tap-ve-nha/buoi-07-sso-va-kiem-thu.html` (~30 phút, nộp trước buổi 8)